# EECE 7205. Fundamentals of Computer Engineering.
# Fall 2026
# In-Class Work: Deleting from a Binary Search Tree

Before beginning your work, read this document from start to finish. It has three parts. **Learning Objectives** lists what you should be able to do by the end of the class. **Background** explains the deletion rules and the five cases, using the programs case1.cpp to case5.cpp that we ran in the lecture. **Tasks** gives four questions for each case. Work through the cases in order, because each one builds on the one before it.

**Understanding the Binary Search Tree Property**

- Explain why every value in a node's left subtree is smaller than the node, and every value in its right subtree is larger.
- Use an inorder traversal to check that a tree is still a valid binary search tree after a change.

**Applying the Deletion Rules**

- Search for a key while remembering its parent, and recognise which of the five cases applies.
- Delete a leaf, a node with one child and a node with two children, including the root.

**Working with the C++ Programs**

- Trace the pointer changes in each program and predict its output before running it.
- Modify the programs to delete other keys, to use the inorder predecessor, and to handle an empty tree.

**Analysing the Cost**

- Relate the number of comparisons to the height h of the tree, and explain when deletion is fast and when it is slow.

## In-Class Work Instructions

- **Work in pairs.** One person reads the code aloud while the other traces the pointers on paper, then swap roles for the next case.
- **Draw first, run second.** For every question, draw the tree and write your predicted output before you run the program. Then run it and compare.
- **Use the provided programs.** Start from case1.cpp to case5.cpp. When a question asks you to modify a program, save it under a new name, for example case2_q2.cpp.
- **Show your work.** Hand in your drawings, your answers and screenshots of each run at the end of the class.

## Background: The Deletion Rules

**Why deletion needs its own rules.** In a binary search tree every value in the left subtree of a node is smaller than the node, and every value in its right subtree is larger. Deleting a node must keep this property for every node that stays in the tree. That is why we cannot simply move the deepest node into the gap, as we did for the level-order binary tree. Instead, we look at how many children the node has and pick one of the rules below.

| Rule | What it says |
|---|---|
| B1 Search | Start at the root and remember the parent. If key < node go left, if key > node go right. |
| B2 Not found | If the search reaches NULL, the key is not in the tree and nothing changes. |
| B3 Leaf | The node has no children: set the parent's link to NULL and free the node. |
| B4 One child | The only child takes the node's place: the parent now points to the child. |
| B5 Two children | Find the inorder successor (right once, then left as far as possible), copy its value into the node, then remove the successor with B3 or B4. |
| B6 Root | The root follows the same rules. With 0 or 1 child, the root pointer itself changes. |
| B7 Check | After every deletion, the inorder traversal must still be in increasing order. |

**The two trees.** Tree T is the tree used in the programs. Tree P is built by inserting 45, 25, 65, 15, 35, 55, 75, 30, 60 in that order with the usual BST insert. Unlike T, tree P has nodes with exactly one child, so it gives us more cases to practise on.

```
Tree T (case1.cpp to case5.cpp)

          40
        /    \
      20      60
     /  \    /  \
   10   30  50   70

Inorder: 10 20 30 40 50 60 70
```

```
Tree P (insert 45, 25, 65, 15, 35, 55, 75, 30, 60)

            45
          /    \
        25      65
       /  \    /  \
     15   35  55   75
          /      \
        30        60

Inorder: 15 25 30 35 45 55 60 65 75
```

**The shared code.** All five programs share the same Node class and two helper functions. Only the part between Before and After changes from case to case.

```
class Node
{
public:
    int data;
    Node* left;
    Node* right;

    Node(int value)
    {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

// Inorder (Left, Node, Right) prints a BST in sorted order
void printInorder(Node* node)
{
    if (node == nullptr) return;
    printInorder(node->left);
    cout << node->data << " ";
    printInorder(node->right);
}

// Free every node (postorder: children before parent)
void freeTree(Node* node)
{
    if (node == nullptr) return;
    freeTree(node->left);
    freeTree(node->right);
    delete node;
}
```

## Tasks:

### Case 1: Deleting the Root (case1.cpp)

**Description:** The root is the first node of the tree, so deleting it is the binary search tree version of "delete at the beginning". In tree T the root 40 has two children, so we cannot just remove it: both subtrees would lose their parent. Instead we find the **inorder successor**, the smallest value that is larger than 40. It is always in the right subtree: go right once (60), then left as far as possible (50). We copy 50 into the root and remove the old node 50, which is a leaf. The root keeps its two subtrees, and 50 is still larger than everything on the left and smaller than everything on the right.

**Rules used:** B5 and B6.

```
// Case 1: delete the root (40). It has two children.
// 1. find the inorder successor: the smallest value in the right subtree
Node* parent = root;
Node* succ = root->right;              // go right once (60)
while (succ->left != nullptr)          // then left as far as possible
{
    parent = succ;
    succ = succ->left;                 // succ = 50, parent = 60
}
// 2. copy the successor's value into the root
root->data = succ->data;               // root now holds 50
// 3. remove the successor node (it has no left child)
if (parent == root)
    parent->right = succ->right;
else
    parent->left = succ->right;        // 60->left = NULL
delete succ;
```

**Question:** Answer the following using tree T and tree P.

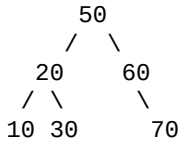
1. In tree T, list every node visited while looking for the inorder successor of the root. Give the successor's value and the value of its parent.
2. Draw tree T after deleting 40. Write its inorder and preorder traversals, and explain how the

Visited: 60 -> 50. Successor = 50; successor parent = 60.

inorder result shows that rule B7 still holds.

3. Modify case1.cpp to use the inorder **predecessor** instead: the largest value in the left subtree (go

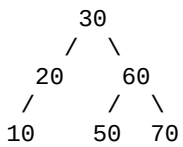
Replace 40 with 50; remove the original 50. Inorder: 10 20 30 50 60 70. Preorder: 50 20 10 30 60 70. Sorted inorder verifies B7.




left once, then right as far as possible). What is the new root, and does the inorder output change? Why or why not?

4. In tree P, delete the root 45. Which node is the successor? Which rule removes the successor node,

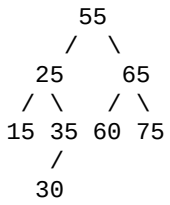
Predecessor = 30 (left once, then right). New root = 30. Inorder remains 10 20 30 50 60 70 because both methods remove only key 40.




and why is it not B3 this time? Draw the tree after the deletion.

The output should look something like this (replace x with your results):

Successor = 55, which has a right child 60. Remove the original 55 by B4 (one child), not B3 (leaf).




```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  x x x x x x   (root = x)
```

Ensure your answers include your tree drawings, the lines of code you changed, and execution snapshots.

## Case 2: Deleting a Leaf (case2.cpp)

**Description:** A **leaf** has no children, so nothing hangs below it. Removing it cannot break the order of any other node. The only thing to do is find it, remember its parent while searching (rule B1), set the parent's link to NULL, and free the node. The program searches for 10: 10 < 40 so it goes left, 10 < 20 so it goes left again, and finds 10 with parent 20. Because 10 is 20's left child, it sets 20->left to NULL.

**Rules used:** B1 and B3.

```
// Case 2: delete a leaf (10). It has no children.
// 1. search for 10, remembering its parent
Node* parent = nullptr;
Node* cur = root;
int key = 10;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
// 2. unlink it from its parent (20) and free it
if (parent->left == cur) parent->left = nullptr;
else                     parent->right = nullptr;
delete cur;
```

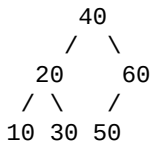
**Question:** Answer the following using tree T and tree P.

1. Trace the search loop for key = 10 in tree T. For each node visited, write the comparison made and the direction taken. Which node is the parent, and is the deleted node its left or its right child?

40: 10 < 40 -> left; 20: 10 < 20 -> left; 10: found. Parent = 20; deleted node is its left child.
Set 20->left = nullptr.

2. Change key to 70 and run the program again. Which pointer is set to NULL now, and what is

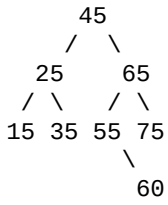
Search 40 -> 60 -> 70. Set 60->right = nullptr. After: 10 20 30 40 50 60 (root = 40).




printed after the deletion?

3. List all the leaves of tree P. Delete 30 from P, draw the tree after the deletion, and name the node

Leaves of original P: 15, 30, 60, 75. Delete 30; 35 becomes a leaf. Inorder: 15 25 35 45 55 60 65 75.




that has just become a leaf.

4. How many key comparisons does the search make in the worst case for a tree of height h? What is h for tree T, and what is h for the tree you get by inserting 10, 20, 30, 40, 50, 60, 70 in that order? What does this tell you about the time to delete a leaf?

At most h + 1 visited nodes/comparisons if height h counts edges. T: h = 2. Sorted insertion 10,20,...,70: h = 6 (right-skewed). Search/deletion O(h): balanced O(log n), skewed O(n).

The output should look something like this (replace x with your results):

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  x x x x x x   (root = 40)
```

Ensure your answers include your tree drawings, the lines of code you changed, and execution snapshots.

## Case 3: Deleting a Node with One Child (case3.cpp)

**Description:** When a node has **exactly one child**, that child (together with everything below it) simply moves up into the node's place. This is safe because the whole child subtree was already on the correct side of the parent: if the node was the parent's right child, every value below it is larger than the parent, and that is still true after it moves up. In tree T every node has 0 or 2 children, so case3.cpp first removes 50 to give 60 a single child, 70. Then it deletes 60 and links 40->right directly to 70.

**Rules used:** B1 and B4 (and B6 if the node is the root).

```
    // Case 3: delete a node with ONE child.
    // First remove 50 so that 60 has only one child (70).
    delete root->right->left;
    root->right->left = nullptr;
    cout << "Middle: ";
    printInorder(root);
    cout << "  (60 now has one child, 70)" << endl;

    // 1. search for 60, remembering its parent
    Node* parent = nullptr;
    Node* cur = root;
    int key = 60;
    while (cur != nullptr && cur->data != key)
    {
        parent = cur;
        if (key < cur->data) cur = cur->left;
        else                 cur = cur->right;
    }
    // 2. its only child takes its place
    Node* child = (cur->left != nullptr) ? cur->left : cur->right;   // 70
    if (parent->left == cur) parent->left = child;
    else                     parent->right = child;                 // 40->right = 70
    delete cur;
```

**Question:** Answer the following using tree P unless a question says otherwise.

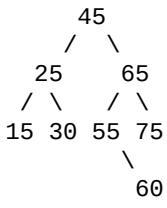
1. Which nodes of tree P have exactly one child? For each one, say whether the child is a left or a right child.
2. Delete 35 from P. Name the parent, the child that moves up, and the exact pointer that changes.

35 has one left child (30); 55 has one right child (60).

Draw the tree and write its inorder traversal.

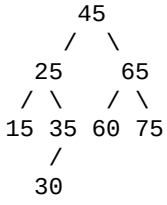
3. Start again from the original P and delete 55. Answer the same three parts as in question 2. Why

Parent = 25; promoted child = 30; change 25->right to node 30. Inorder: 15 25 30 45 55 60 65 75.




does the program use parent->left this time instead of parent->right?

From original P: parent = 65; promoted child = 60; change 65->left to node 60, because 55 is the left child of 65. Inorder: 15 25 30 35 45 60 65 75.



4. Build a tree by inserting 10, 20, 30 in that order and delete the root 10. Which rule tells you that the root pointer itself must change? What is the new root, and what goes wrong in case3.cpp if you use it unchanged (hint: what is parent when the root is deleted)?

Inserting 10,20,30 creates 10 ->right 20 ->right 30. Deleting root 10 promotes 20. Rule B6 updates root. In case3.cpp, parent is nullptr for root, so dereferencing parent->left/right is invalid.

```
Before: 10 -> 20 -> 30 (right links)
After:  20 -> 30 (right link)
```

The output should look something like this (replace x with your results):

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  x x x x x x x x x   (root = 45)
```

Ensure your answers include your tree drawings, the lines of code you changed, and execution snapshots.

## Case 4: Deleting a Node with Two Children (case4.cpp)

**Description:** A node with **two children** cannot be replaced by one of them, because the other subtree would lose its place. We use the same idea as in Case 1: copy in the value of the **inorder successor** and then delete the successor node instead. To delete 20 in tree T, the successor is found by going right once (30) and then left as far as possible (30 has no left child, so we stop). 30 is copied into the node, and the old 30 is removed. Because the successor is the leftmost node of a subtree, it never has a left child, so removing it is always an easy B3 or B4 case.

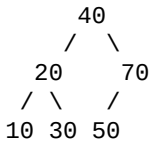
**Rules used:** B1 and B5 (then B3 or B4 for the successor).

```
// Case 4: delete a node with TWO children (20).
// 1. search for 20
Node* cur = root;
int key = 20;
while (cur != nullptr && cur->data != key)
{
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
// 2. find the inorder successor: right once, then left as far as possible
Node* parent = cur;
Node* succ = cur->right;              // 30
while (succ->left != nullptr)
{
    parent = succ;
    succ = succ->left;
}
// 3. copy the successor's value into the node
cur->data = succ->data;               // the node that held 20 now holds 30
// 4. remove the successor node (it has no left child)
if (parent == cur)
    parent->right = succ->right;      // 30 was cur's right child
else
    parent->left = succ->right;
delete succ;
```

**Question:** Answer the following using tree T and tree P.

1. Change key to 60 in case4.cpp. What are succ and parent after the loop? Which branch of the final if statement runs, and why? What is printed after the deletion?
2. Explain in your own words why the inorder successor of a node with two children can never have

For key 60: succ = 70 and parent = 60; the successor-search loop is skipped. The parent == cur branch runs, setting parent->right = succ->right (nullptr). After: 10 20 30 40 50 70.




a left child. What would go wrong in step 4 of the code if it could?

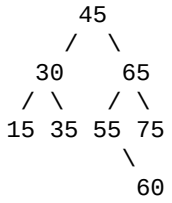
3. In tree P, delete 25. Give the successor, its parent, and the pointer that is changed. Draw the tree

The successor is the smallest key in the right subtree. A left child would be even smaller, contradicting this definition. Removing such a successor without preserving that left subtree would lose nodes.

after the deletion and write its inorder traversal.

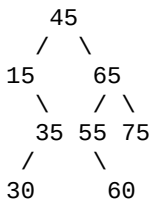
4. Delete 25 from the original P again, but this time use the inorder predecessor. Draw the result. Is it

Successor = 30; its parent = 35. Copy 30 into node 25 and set 35->left = nullptr. Inorder: 15 30 35 45 55 60 65 75.




the same tree as in question 3? Are both valid binary search trees?

Predecessor = 15. Replace 25 with 15, then remove original leaf 15. Different shape than Q3; both are valid BSTs. Inorder: 15 30 35 45 55 60 65 75.




The output should look something like this (replace x with your results):

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  x x x x x x   (root = 40)
```

Ensure your answers include your tree drawings, the lines of code you changed, and execution snapshots.

## Case 5: The Key Is Not in the Tree (case5.cpp)

**Description:** Before deleting anything we must find the key. The search follows the BST property: at each node it goes left if the key is smaller and right if it is larger. If it reaches **NULL**, the key is not in the tree and the tree must be left exactly as it was. For 25 in tree T the path is 40 (go left), 20 (go right), 30 (go left), then NULL. The NULL position the search stops at is exactly where 25 would go if we inserted it, which is why insertion and search share the same loop.

**Rules used:** B1 and B2.

```
    // Case 5: the key is not in the tree (delete 25).
    Node* cur = root;
    int key = 25;
    while (cur != nullptr && cur->data != key)
    {
        if (key < cur->data) cur = cur->left;    // 25 < 40: go left to 20
        else                 cur = cur->right;   // 25 > 20: go right to 30
    }                                            // 25 < 30: go left, NULL
    if (cur == nullptr)
        cout << key << " is not in the tree: nothing to delete" << endl;
```

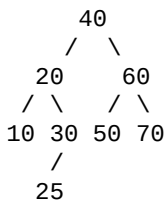
**Question:** Answer the following using tree T.

1. Trace the search for 25. How many comparisons with node values does it make, and at which NULL link (which node, left or right) does it stop?
2. Do the same for the keys 5, 45 and 75. Which one needs the most comparisons, and how does that relate to the height of the tree?
3. Modify case5.cpp so that when the key is not found, it inserts the key at the NULL link where the search stopped (you will need to remember the parent). Insert 25 and print the inorder traversal.
4. case5.cpp prints root->data. What happens if the tree is empty (root = nullptr)? Add a check so the program prints "the tree is empty" instead. Which rule covers this case?

40 -> 20 -> 30 -> NULL at 30->left. Three node-value comparisons: 25 < 40, 25 > 20, 25 < 30.

5: 40 -> 20 -> 10 -> left NULL (3). 45: 40 -> 60 -> 50 -> left NULL (3). 75: 40 -> 60 -> 70 -> right NULL (3). All tie. Worst-case comparisons O(h).

Remember parent during search; when 25 is missing, attach a new Node(25) at 30->left. Inorder: 10 20 25 30 40 50 60 70.




On an empty tree, root->data dereferences nullptr (undefined behavior). Check root first: if (!root) cout << "the tree is empty"; else cout << root->data; Rule B2 covers not found; B6 covers root changes.

The output should look something like this (replace x with your results):

```
Before: 10 20 30 40 50 60 70   (root = 40)
x is not in the tree: nothing to delete
After:  10 20 30 40 50 60 70   (root = 40)
```

Ensure your answers include your tree drawings, the lines of code you changed, and execution snapshots.

BONUS: Use the unified deleteNode(root,key) function in the attached C++ file. Return the new subtree root in all cases: not found, leaf, one child, two children (successor), and root. Sequence 40,10,60,25 gives the traversals below. O(h) time, O(h) recursive stack.

```
Initial:   10 20 30 40 50 60 70 (root 40)
Delete 40: 10 20 30 50 60 70 (root 50)
Delete 10: 20 30 50 60 70 (root 50)
Delete 60: 20 30 50 70 (root 50)
Delete 25: 20 30 50 70 (root 50)

Final:      50
           /  \
          20   70
            \
             30
```

**Notes:**

If you have any questions while working on this in-class work, please raise your hand or ask during the session.

**Execution evidence (user-provided terminal screenshot)**

```
./bst_all_cases T delete 40

# Case 2: Delete leaf 70
./bst_all_cases T delete 70

# Case 3: Delete node 35 from Tree P
./bst_all_cases P delete 35

# Case 4: Delete node 25 from Tree P
./bst_all_cases P delete 25

# Case 5: Insert missing key 25
./bst_all_cases T insert 25

# Test an empty tree
./bst_all_cases empty
Before: 10 20 30 40 50 60 70  (root = 40)
After: 10 20 30 50 60 70  (root = 50)
Preorder: 50 20 10 30 60 70
Before: 10 20 30 40 50 60 70  (root = 40)
After: 10 20 30 40 50 60  (root = 40)
Preorder: 40 20 10 30 60 50
Before: 15 25 30 35 45 55 60 65 75  (root = 45)
After: 15 25 30 45 55 60 65 75  (root = 45)
Preorder: 45 25 15 30 65 55 60 75
Before: 15 25 30 35 45 55 60 65 75  (root = 45)
After: 15 30 35 45 55 60 65 75  (root = 45)
Preorder: 45 30 15 35 65 55 60 75
Before: 10 20 30 40 50 60 70  (root = 40)
After: 10 20 25 30 40 50 60 70  (root = 40)
Preorder: 40 20 10 30 25 60 50 70
the tree is empty
```

## Appendix: Complete C++ source code

```
#include <iostream>
#include <string>
using namespace std;
struct Node { int data; Node *left, *right; Node(int v):data(v),left(nullptr),right(nullptr){} };
Node* insert(Node* r,int k){if(!r)return new Node(k);if(k<r->data)r->left=insert(r->left,k);else if(k>r->data)
Node* makeTree(char t){Node* r=nullptr;int T[]={40,20,60,10,30,50,70};int P[]={45,25,65,15,35,55,75,30,60};if(
void printInorder(Node* r){if(!r)return;printInorder(r->left);cout<<r->data<<' ';printInorder(r->right);}
void printPreorder(Node* r){if(!r)return;cout<<r->data<<' ';printPreorder(r->left);printPreorder(r->right);}
void freeTree(Node* r){if(!r)return;freeTree(r->left);freeTree(r->right);delete r;}
// B1-B6: deletion, returning possibly updated root. O(h) time.
Node* deleteNode(Node* root,int key){
 if(!root)return nullptr;
 if(key<root->data){root->left=deleteNode(root->left,key);return root;}
 if(key>root->data){root->right=deleteNode(root->right,key);return root;}
 if(!root->left){Node* child=root->right;delete root;return child;}
 if(!root->right){Node* child=root->left;delete root;return child;}
 Node* succ=root->right;
 while(succ->left)succ=succ->left;
 root->data=succ->data;
 root->right=deleteNode(root->right,succ->data);
 return root;
}
Node* deletePredecessor(Node* root,int key){
 if(!root)return nullptr;
 if(key<root->data){root->left=deletePredecessor(root->left,key);return root;}
 if(key>root->data){root->right=deletePredecessor(root->right,key);return root;}
 if(!root->left){Node* c=root->right;delete root;return c;}
 if(!root->right){Node* c=root->left;delete root;return c;}
 Node* pred=root->left;while(pred->right)pred=pred->right;
 root->data=pred->data;root->left=deletePredecessor(root->left,pred->data);return root;
}
void show(const string& name,Node* r){cout<<name<<": ";printInorder(r);cout<<" (root = ";if(r)cout<<r->data;el
int main(int argc,char**argv){
 if(argc==1){Node* r=makeTree('T');show("Before",r);for(int k:{40,10,60,25}){r=deleteNode(r,k);show("After del
 string mode=(argc>2?argv[2]:argv[1]);
 if(mode=="empty"){Node* r=nullptr;cout<<"the tree is empty\n";r=deleteNode(r,25);freeTree(r);return 0;}
 if(argc<4){cerr<<"Usage: ./bst_all_cases T|P delete|pred|insert KEY\n";return 1;}
 char t=argv[1][0];if(t!='T'&&t!='P'){cerr<<"Choose T or P\n";return 1;}
 int key=stoi(argv[3]);Node* r=makeTree(t);show("Before",r);
 if(mode=="delete")r=deleteNode(r,key);
 else if(mode=="pred")r=deletePredecessor(r,key);
 else if(mode=="insert")r=insert(r,key);
 else{cerr<<"Unknown mode\n";freeTree(r);return 1;}
 show("After",r);cout<<"Preorder: ";printPreorder(r);cout<<'\n';freeTree(r);
}
```